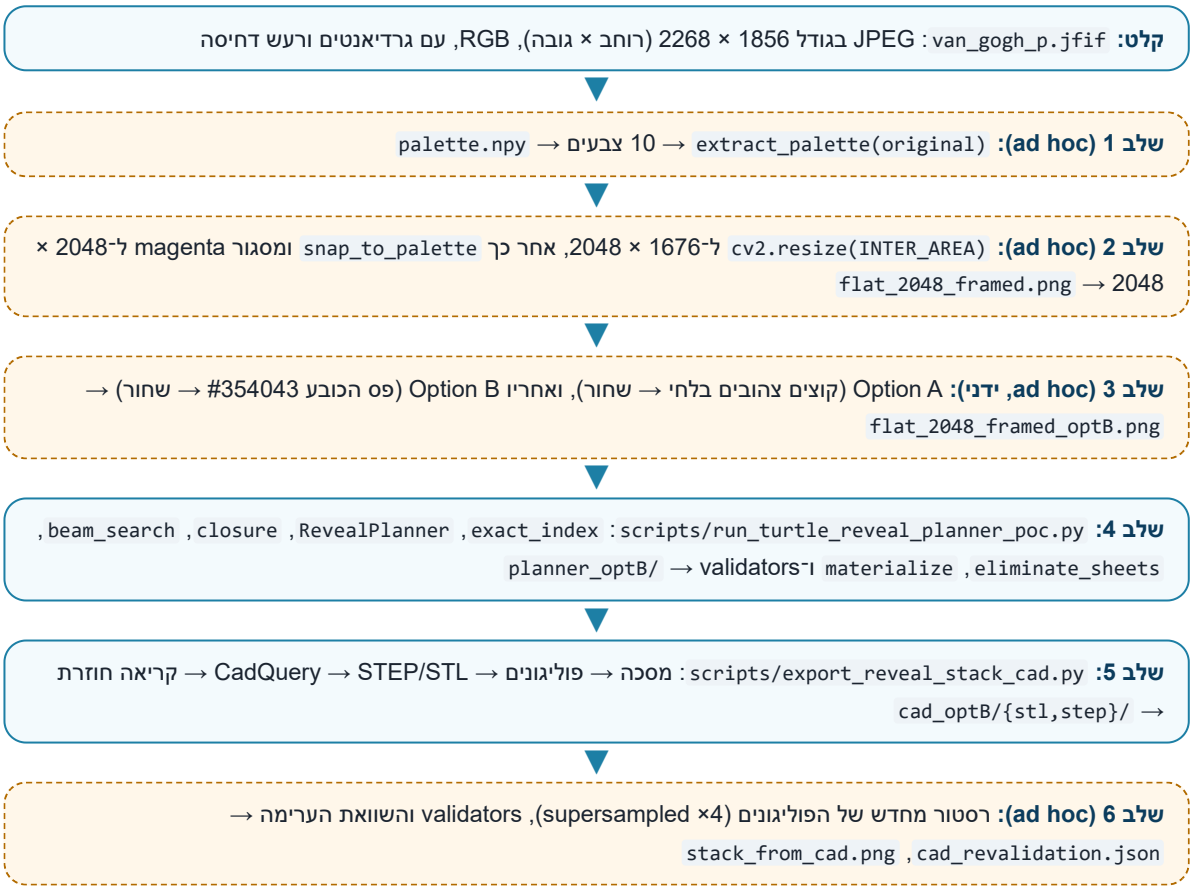
3. נקודות כניסה ורצף הביצוע של Option B

3.1 אין נקודת כניסה אחת

GAP אין במאגר פקודה אחת שהופכת תמונה ל־STL. ל־`python -m layerforge_l1` (`cli.py`) אין קשר למסלול הזה. Option B הופק בשש פקודות. שתיים מהן סקריפטים שנשמרו במאגר, וארבע הן קטעי Python חד־פעמיים שהעוזר הריץ. הקוד המדויק של הקטעים החד־פעמיים מופיע בפרק הזה, כי אין לו עותק אחר במאגר.

3.2 תרשים 2: צינור העיבוד מקלט לפלט



3.3 שלב 1: חילוץ פלטה AD HOC

```
o = np.asarray(Image.open('van_gogh_p.jfif').convert('RGB'))
pal = extract_palette(o)                # k-means in Lab on the full 1856x2268 image
lab = snap_to_palette(o, pal)           # used only for the preview flat_native.png
np.save('artifacts/l1_van_gogh_v1/palette.npy', pal)
```

3.4 שלב 2: הקטנה, מיפוי ומסגור AD HOC

```
pal = np.load('artifacts/l1_van_gogh_v1/palette.npy')
w = round(o.shape[1]*2048/o.shape[0])            # 1676 px = 163.7 mm
small = cv2.resize(o, (w, 2048), interpolation=cv2.INTER_AREA)
lab = snap_to_palette(small, pal)
out = np.full((2048, 2048, 3), (255, 0, 255), np.uint8); x0 = (2048 - w)//2   # x0 = 186
out[:, x0:x0+w] = pal[lab]
Image.fromarray(out).save('artifacts/l1_van_gogh_v1/flat_2048_framed.png')
```

שים לב: הפלטה חושבה ברזולוציה המקורית, אבל המיפוי הסופי בוצע על התמונה המוקטנת. 2048 פיקסלים מייצגים 200 מ"מ, ולכן בתמונת היעד כל פיקסל הוא $200/2048 = 0.09765625$ מ"מ, בדיוק כמו ברשת הפיזית. הסבר מלא בפרק 6.

3.5 שלב 3: תיקוני Option A ו־Option B AD HOC · USER-REQUESTED

```
red = (t == RED).all(-1); left = leftmost 4-connected red component; window = its bbox + 60 px
Y = (t == YELLOW).all(-1)
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (17, 17))          # 17 px = 1.66 mm
thin = Y & ~cv2.morphologyEx(Y, cv2.MORPH_OPEN, k) & window
a = t.copy(); a[thin] = BLACK                                        # Option A
b = a.copy(); b[(b == BAND).all(-1)] = BLACK                         # Option B (BAND = #354043)
```

ההסבר המתמטי מופיע בפרק 4.7. מספרים מהקבצים (OBSERVED_IN_FILE): Option A שינה 173 פיקסלים (1.65 מ"מ²), ו־Option B שינה עוד 13,154 פיקסלים (125.4 מ"מ²).

3.6 שלב 4: המתכנן (סקריפט שנשמר)

```
PYTHONPATH=src .venv/Scripts/python.exe scripts/run_turtle_reveal_planner_poc.py \
  --variants per_cell --skip-x8 --beam 8 --tile-mm 2.0 --score colors \
  --frame-hex "#FF00FF" --target artifacts/l1_van_gogh_v1/flat_2048_framed_optB.png \
  --output artifacts/l1_van_gogh_v1/planner_optB
```

למרות השם "turtle", זה הסקריפט הכללי. היעד מגיע מ־`--target`. סדר הקריאות בתוך `scripts/run_turtle_reveal_planner_poc.py:52`:

scripts/run_turtle_reveal_planner_poc.py:174-189

```
174     target = Path(args.target)
175     rgb = np.asarray(Image.open(target).convert("RGB"))
176     labels, palette = exact_index(rgb)
177     frame_label = None
178     if args.frame_hex:
179         frame_label = [_hex(c) for c in palette].index(args.frame_hex.upper())
180     OUTPUT.mkdir(parents=True, exist_ok=True)
181     results = {"target": str(target.resolve().relative_to(ROOT)), "palette": [_hex(c) for c in palette],
"variants": []}
182     for name in args.variants.split(","):
183         config = RevealConfig(
184             split_regions=name == "per_cell", beam_width=args.beam, planning_margin_pixels=args.margin,
185             cell_tile_mm=args.tile_mm, beam_score=args.score, frame_label=frame_label
186         )
187         results["config"] = config.__dict__
188         results["variants"].append(run_variant(name, labels, palette, config))
189         (OUTPUT / "poc_report.json").write_text(json.dumps(results, indent=2) + "\n", encoding="utf-8")
```

```
scripts/run_turtle_reveal_planner_poc.py:54-60
  54      started = time.perf_counter()
  55      planner = RevealPlanner(labels, config)
  56      row: dict = {"variant": name, "cells": planner.cell_count}
  57      closure = planner.closure()
  58      row["closure_feasible"] = closure["feasible"]
  59      row["closure_round_robin_sheets"] = closure["round_robin_sheet_count"]
  60      row["closure_seconds"] = round(time.perf_counter() - started, 1)
```

```
scripts/run_turtle_reveal_planner_poc.py:76-87
  76      plan = planner.beam_search()
  77      row["search_status"] = plan["status"]
  78      row["search_reveal_evaluations"] = plan["evaluations"]
  79      row["beam_sheet_count"] = len(plan["front_to_back_colors"])
  80      colors, reveals, eliminated = planner.eliminate_sheets(plan["front_to_back_colors"], plan["reveals"])
  81      plan["reveals"] = reveals
  82      row["eliminated_sheets"] = eliminated
  83      print(name, "beam", row["beam_sheet_count"], "after elimination", len(colors), flush=True)
  84      row["sheet_count"] = len(colors)
  85      row["front_to_back_colors"] = [_hex(palette[c]) for c in colors]
  86      sheets = planner.materialize(plan["reveals"])
  87      p = planner.grid.perimeter_pixels
```

3.7 שלב 5: ייצוא CAD (סקריפט שנשמר)

```
PYTHONPATH=src .venv/Scripts/python.exe scripts/export_reveal_stack_cad.py \
  --layers artifacts/l1_van_gogh_v1/planner_optB/per_cell/layers \
  --output artifacts/l1_van_gogh_v1/cad_optB              # defaults: 225 mm, 1.6 mm, simplify 0.03 mm
```

הקבצים ב־`cad_optB` נוצרו לפני שהסקריפט קיבל את מבנה התיקיות `stl/` ו־`step/`. לכן הם הועברו לתיקיות האלה ידנית, ונתיבי ה־manifest עודכנו. הקוד הנוכחי של הסקריפט כבר כותב ישירות למבנה הזה.

3.8 שלב 6: אימות חוזר של ה־CAD AD HOC

קטע הקוד מופיע במלואו בפרק 11.6, כי זה האימות היחיד שבדק שהפוליגונים שנשלחו ל־CAD זהים למסכות. GAP השלב הזה אינו חלק מהסקריפט במאגר.

3.9 זמני ריצה OBSERVED_IN_FILE

שלב	זמן	מקור
closure (בדיקת היתכנות)	15.8 שניות	poc_report.json · closure_seconds
כל המתכנן (closure + beam + materialize + validators + PNG)	304.3 שניות	poc_report.json · total_seconds
ייצוא STEP/STL וקריאה חוזרת	122.9 שניות	cad_manifest.json · total_seconds
אימות חוזר של ה־CAD	10.4 שניות	פלט הקטע החד־פעמי (STATUS.md)